UNIVERSITÄT
LEIPZIG

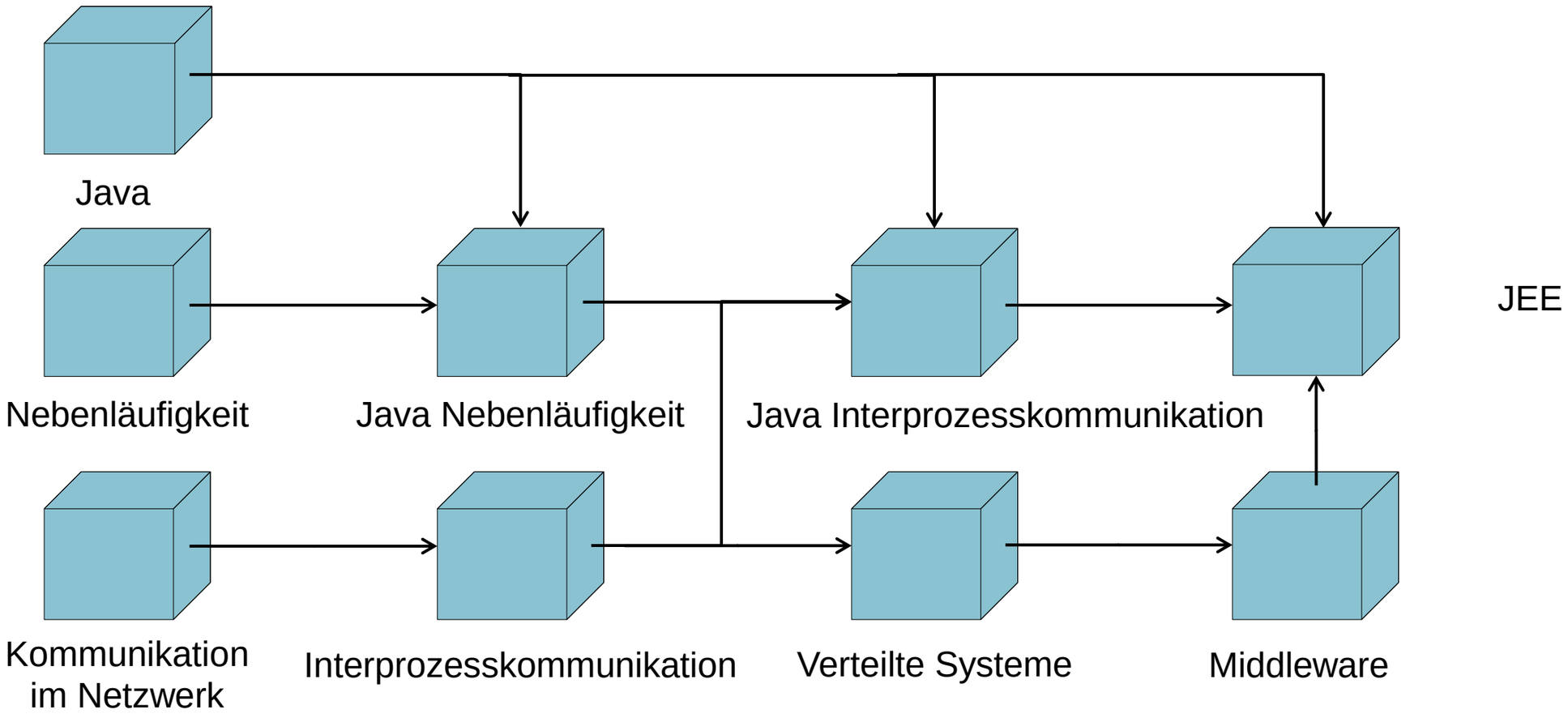
JEE vs Spring

ENTWICKLUNG VERTEILTER ANWENDUNGEN

Leipzig, 23.06.2022

Dr. Pascal Kovacs

AGENDA

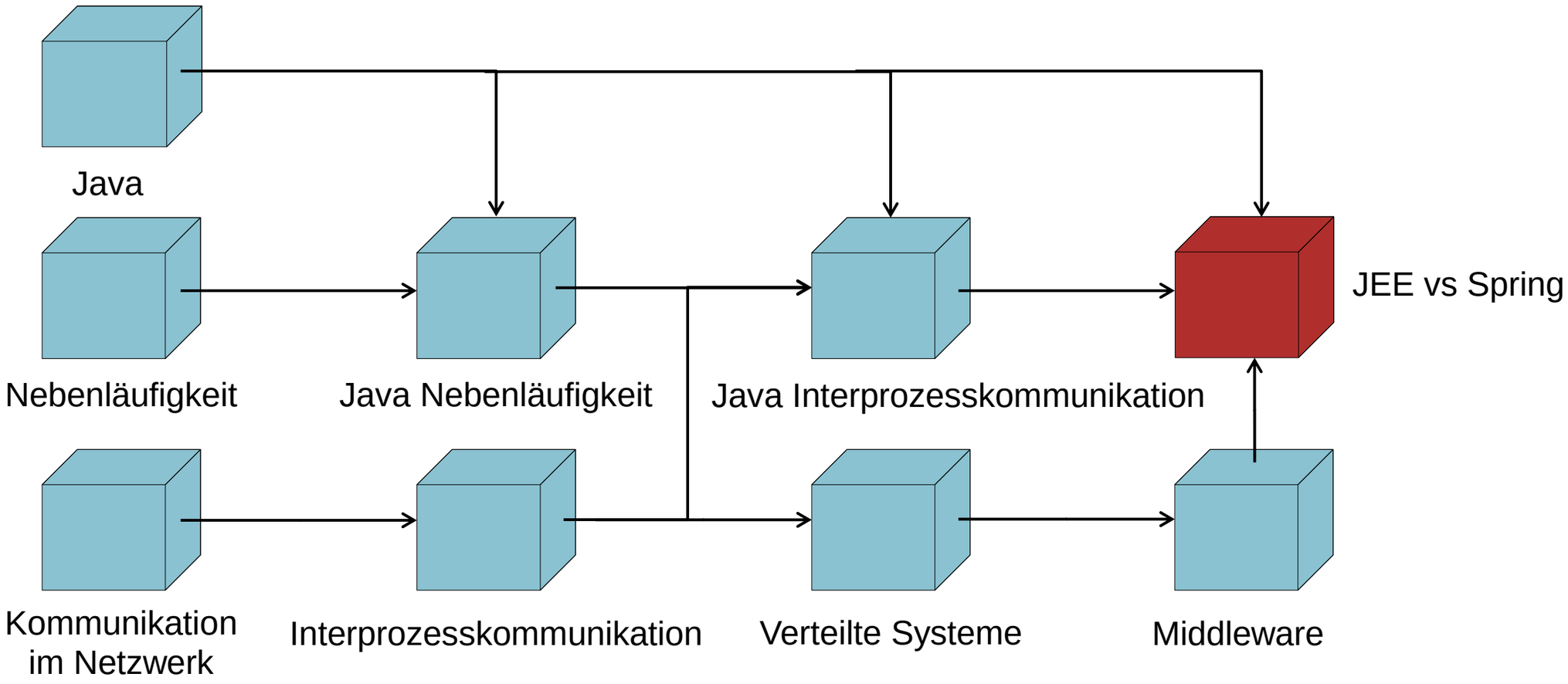

Java
Nebenläufigkeit
Java Nebenläufigkeit
Java Interprozesskommunikation
JEE
Kommunikation im Netzwerk
Interprozesskommunikation
Verteilte Systeme
Middleware

ZIELE

- Aufbau JEE Architektur und Application Server kennen
- Komponentenlebenszyklus und dessen Abbildung verstehen
- ~~Enterprise Java Beans mit Vor- und Nachteilen kennen~~
- ~~Funktionsweise der JPA verstehen~~

BEISPIELFRAGE

AGENDA


Java
Nebenläufigkeit
Java Nebenläufigkeit
Java Interprozesskommunikation
JEE vs Spring
Kommunikation im Netzwerk
Interprozesskommunikation
Verteilte Systeme
Middleware

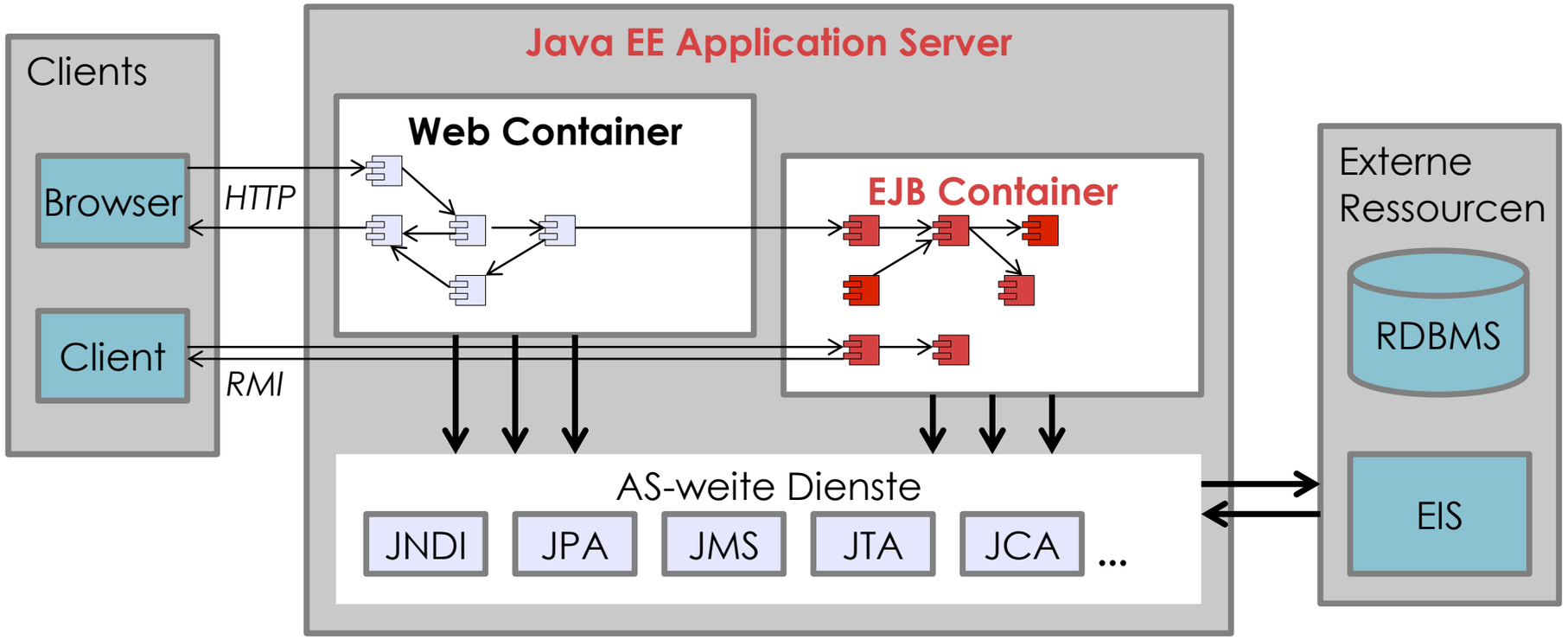
ZIELE

- JEE in der Anwendung verstehen
- Session Beans mit Vor- und Nachteilen kennen
- Anforderungen Zwölf-Faktor Aps verstehen
- Aufbau und Grundlagen von Spring kennen
- Abgrenzung Spring zu JEE verstehen

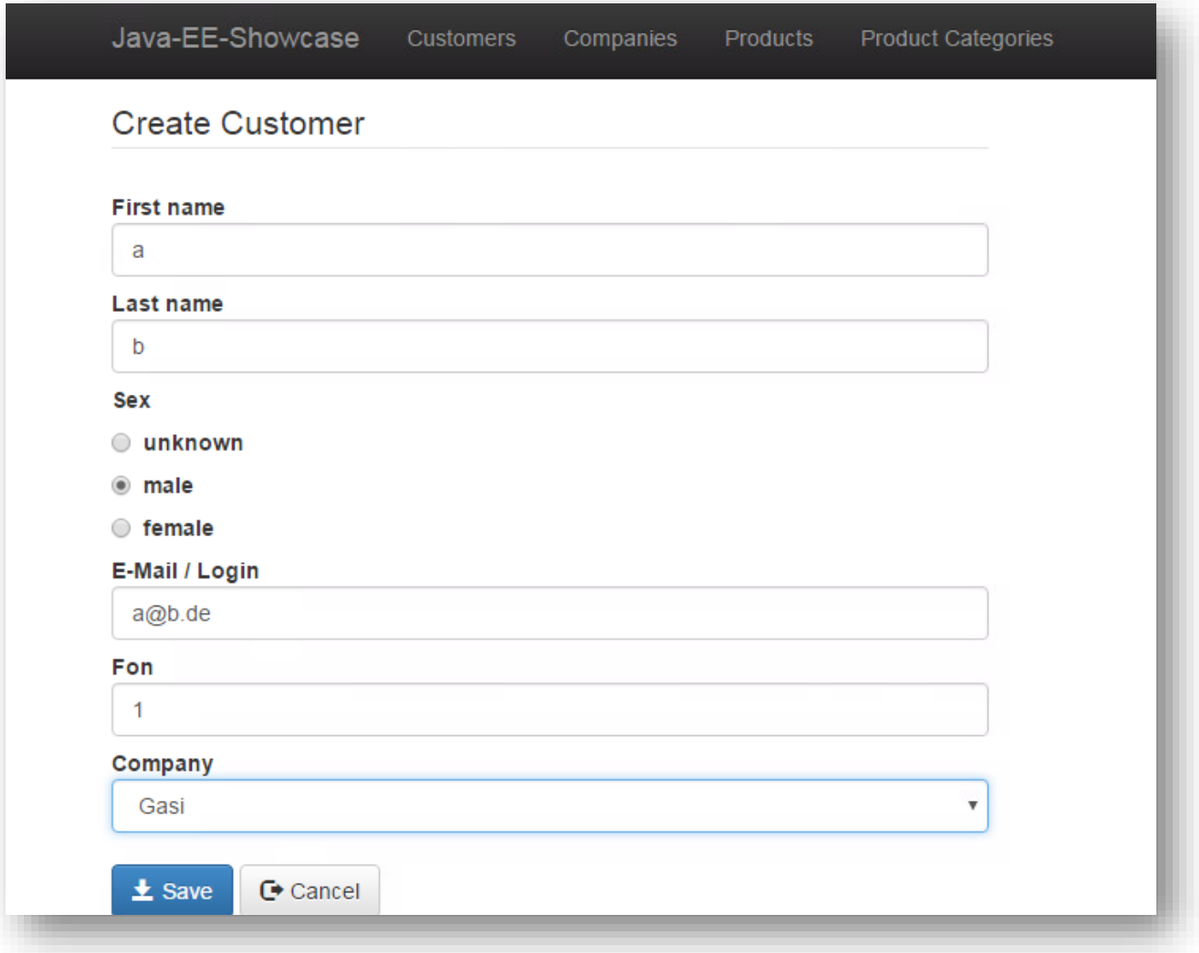
ÜBERSICHT

- Die Grenzen von JEE
- 12 Faktor Apps
- Spring

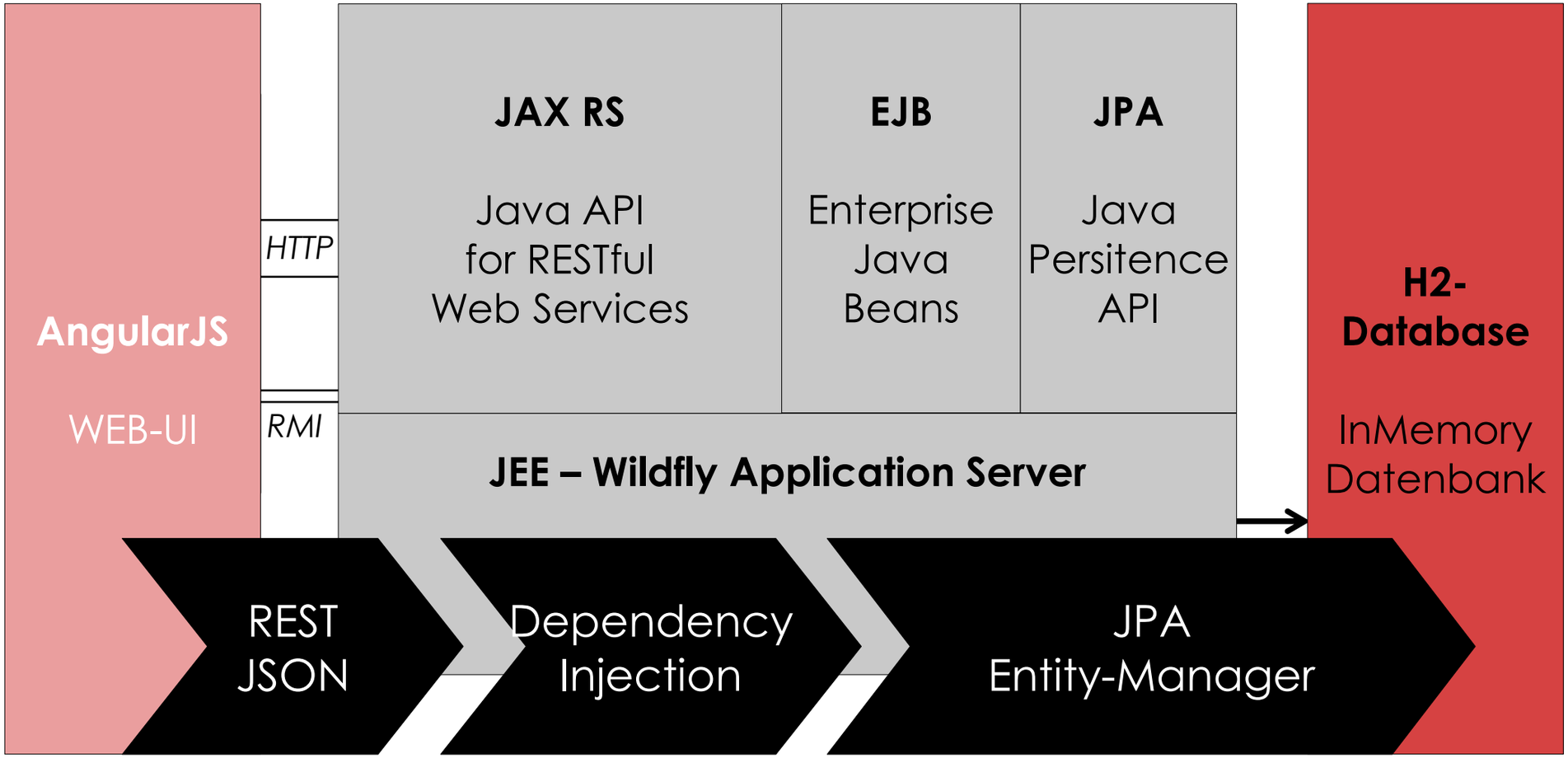
JEE ARCHITEKTURBILD



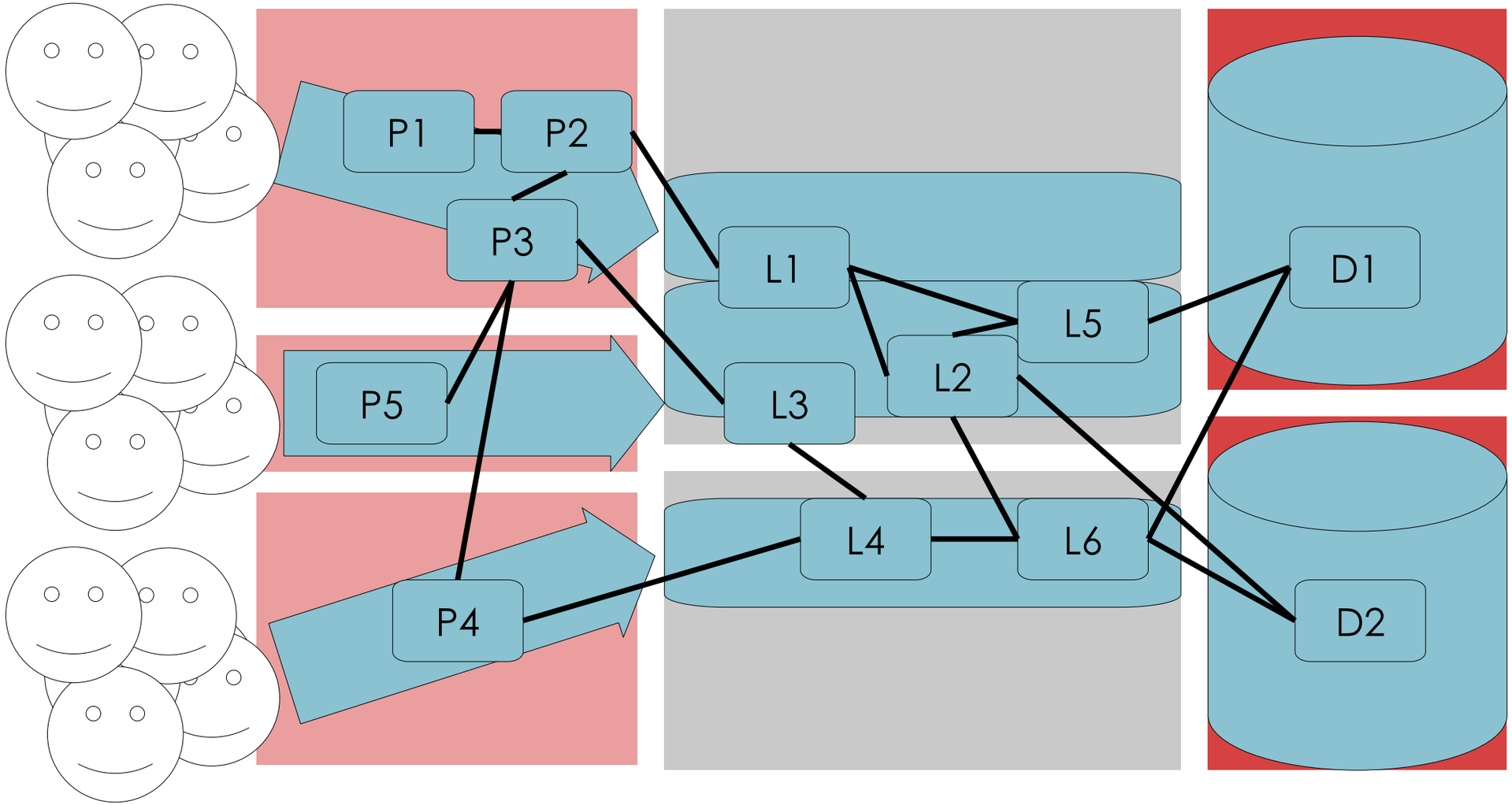
REALISIERUNG ROBUSTER, ERWEITERBARER, SKALIERBARER, KOMPONENTENBASIERTER… VERTEILTER! ANWENDUNGEN



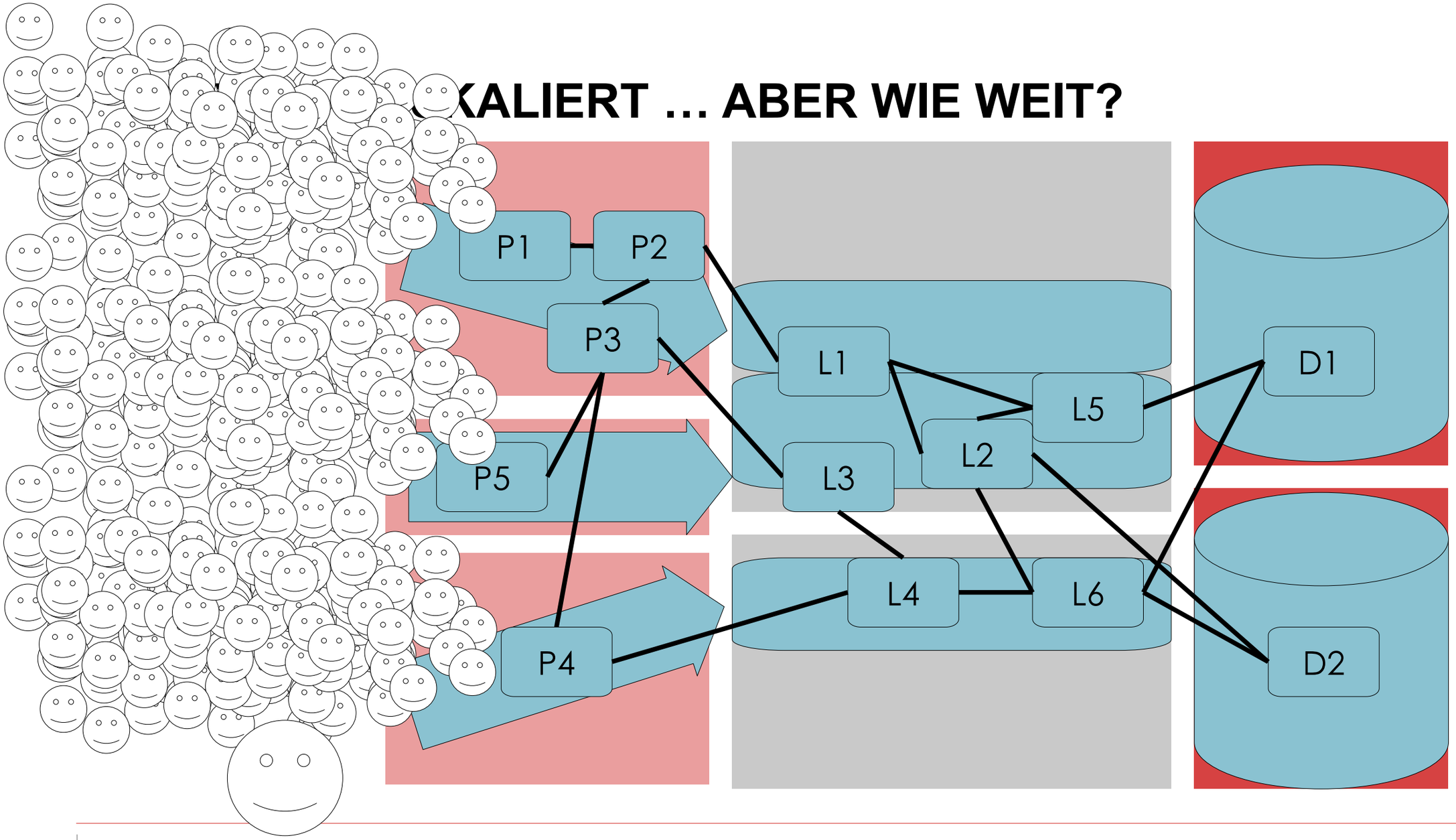
ARCHITEKTUR



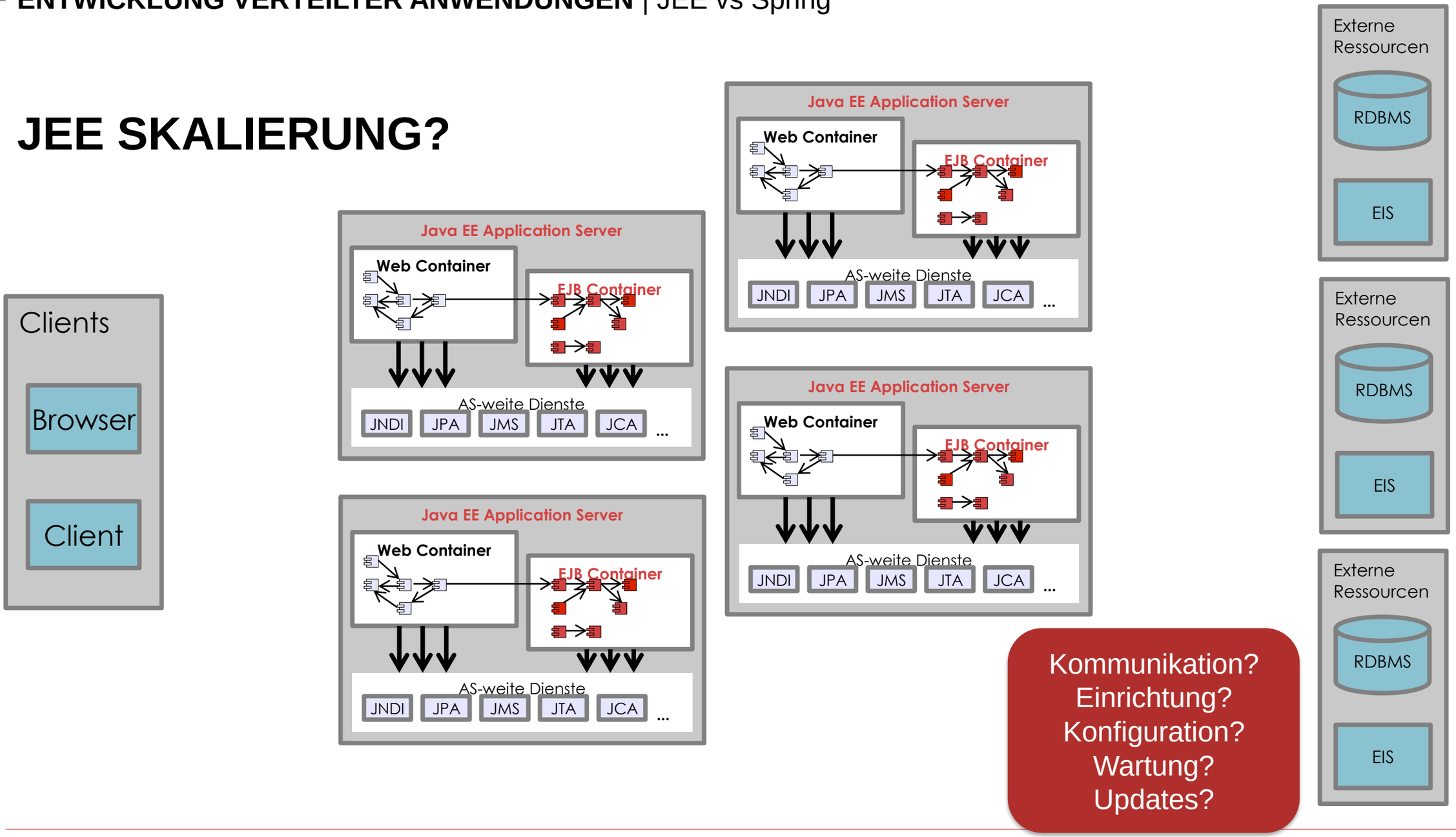
TICKETSHOP SKALIERT …



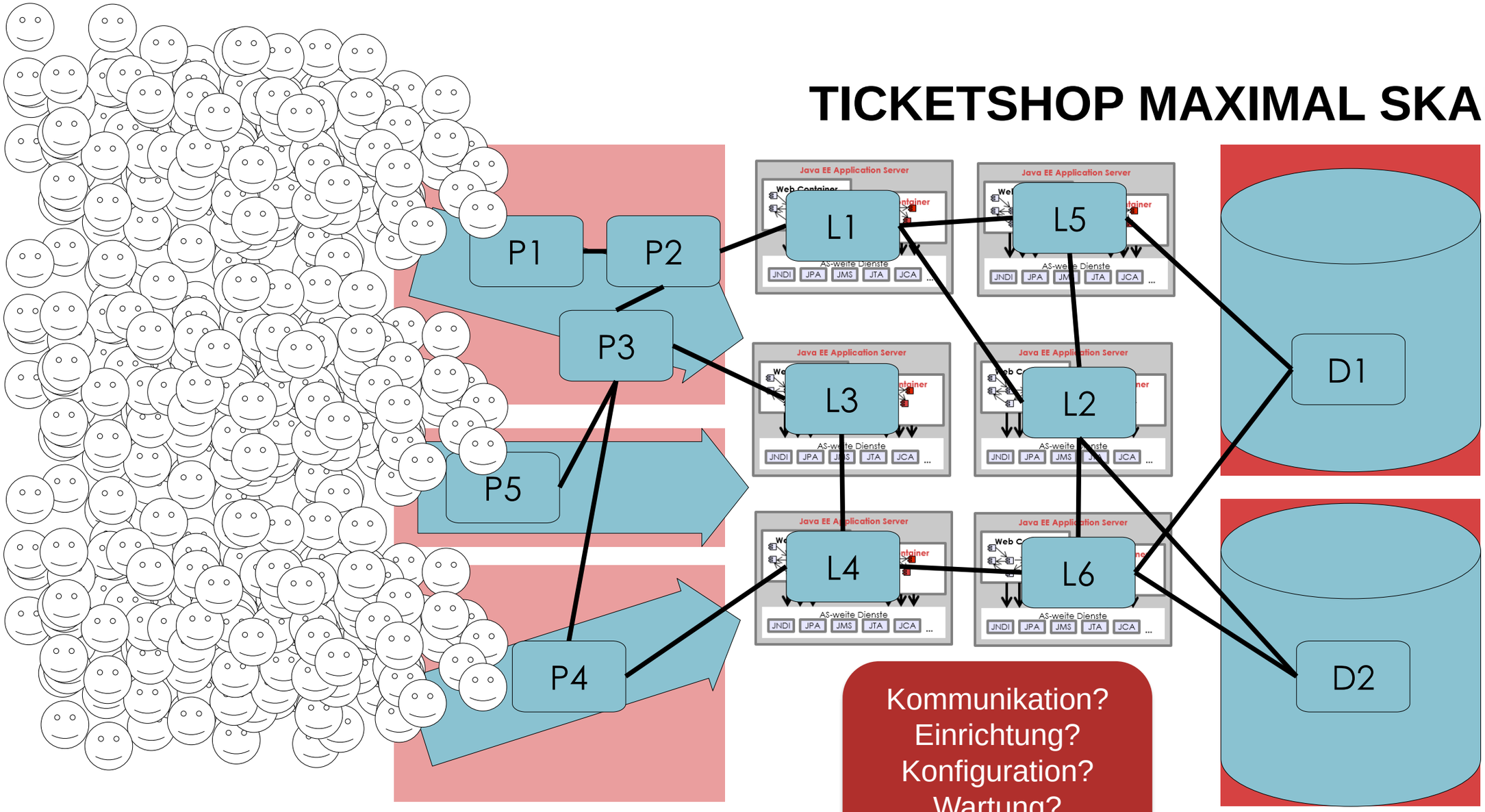
KALIERT … ABER WIE WEIT?



JEE SKALIERUNG?

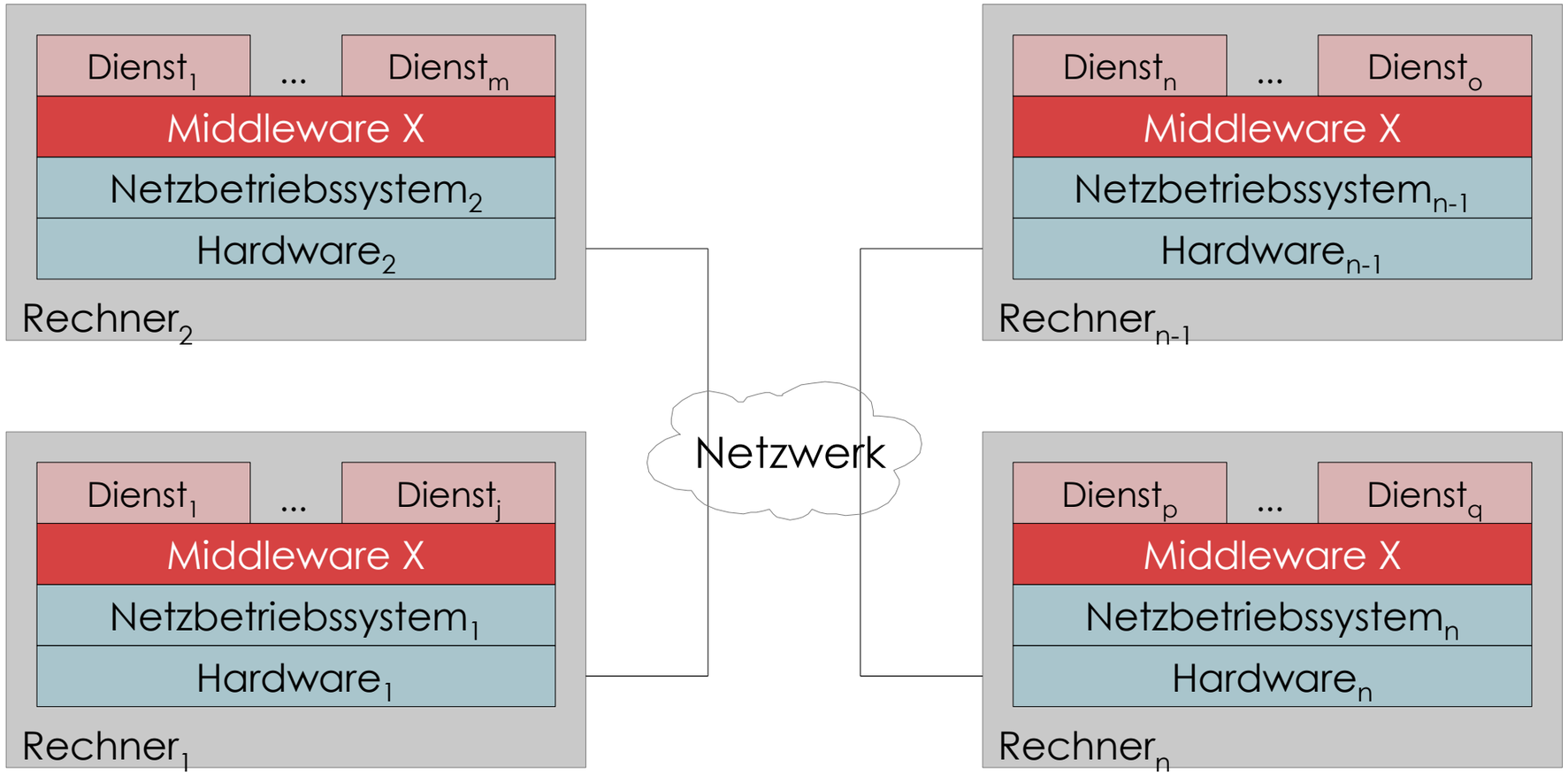


TICKETSHOP MAXIMAL SKALIERT?



Kommunikation?
Einrichtung?
Konfiguration?
Wartung?
Updates?

MODELL EINES VERTEILTEN SYSTEMS

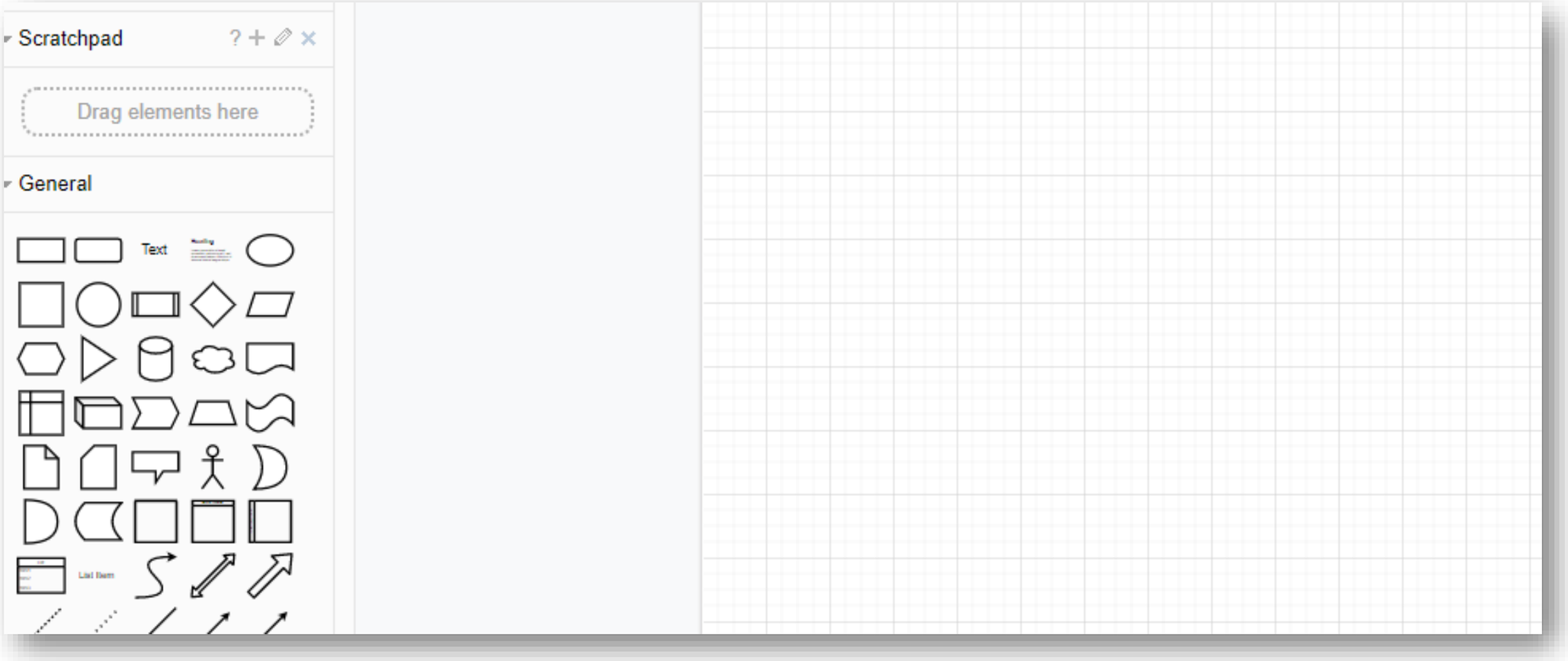


Ähnlich zu: Emmerich, W.: Konstruktion von verteilten Objekten, S. 9

ÜBERSICHT

- JEE am Beispiel CRM-DEMO
- 12 Faktor Apps
- Spring

HOCH SKALIERBARE FUNKTIONALE WEB APPLICATIONEN



[draw.io - app.diagrams.net]

TWELVE FACTOR APPS - 12FACTOR.NET

- Methode zur Entwicklung von *Software-As-A-Service* (SAAS) Apps, die
  - Deklarative Formate für die Automatisierung der Konfiguration nutzt
  - Sauberen Vertrag mit dem zugrundeliegenden Betriebssystem hat
  - Deployment auf modernen Cloud-Plattformen unterstützt
  - Minimale Abweichung zwischen Entwicklungs- und Produktionsumgebung hat
  - Hochgradig skaliert
- Vorteile
  - Ersparnis von Zeit und Kosten für die Entwicklung
  - Maximale Portierbarkeit, Kein Server und keine Serveradministration notwendig
  - Maximales agiles entwickeln möglich
  - Keine wesentliche Änderungen in Tooling, Architektur oder Entwicklungsverfahren

[https://blog.viadee.de/java-ee-ist-tot-es-lebe-spring]

DIE ZWÖLF FAKTOREN

1. *Codebase* – Eine im Versionsmanagementsystem verwaltete Codebase, viele Deployments
2. *Abhängigkeiten* – Abhängigkeiten explizit deklarieren und isolieren
3. *Konfiguration* – Die Konfiguration in Umgebungsvariablen ablegen
4. *Unterstützende Dienste* – Unterstützende Dienste als angehängte Ressourcen behandeln
5. *Build, release, run* – Build- und Run-Phase strikt trennen
6. *Prozesse* – Die App als einen oder mehrere Prozesse ausführen
7. *Bindung an Ports* – Dienste durch das Binden von Ports exportieren
8. *Nebenläufigkeit* – Mit dem Prozess-Modell skalieren
9. *Einweggebrauch* – Robuster mit schnellem Start und problemlosen Stopp
10. *Dev-Prod-Vergleichbarkeit* – Entwicklung, Staging und Produktion so ähnlich wie möglich halten
11. *Logs* – Logs als Strom von Ereignissen behandeln
12. *Admin-Prozesse* – Admin/Management-Aufgaben als einmalige Vorgänge behandeln

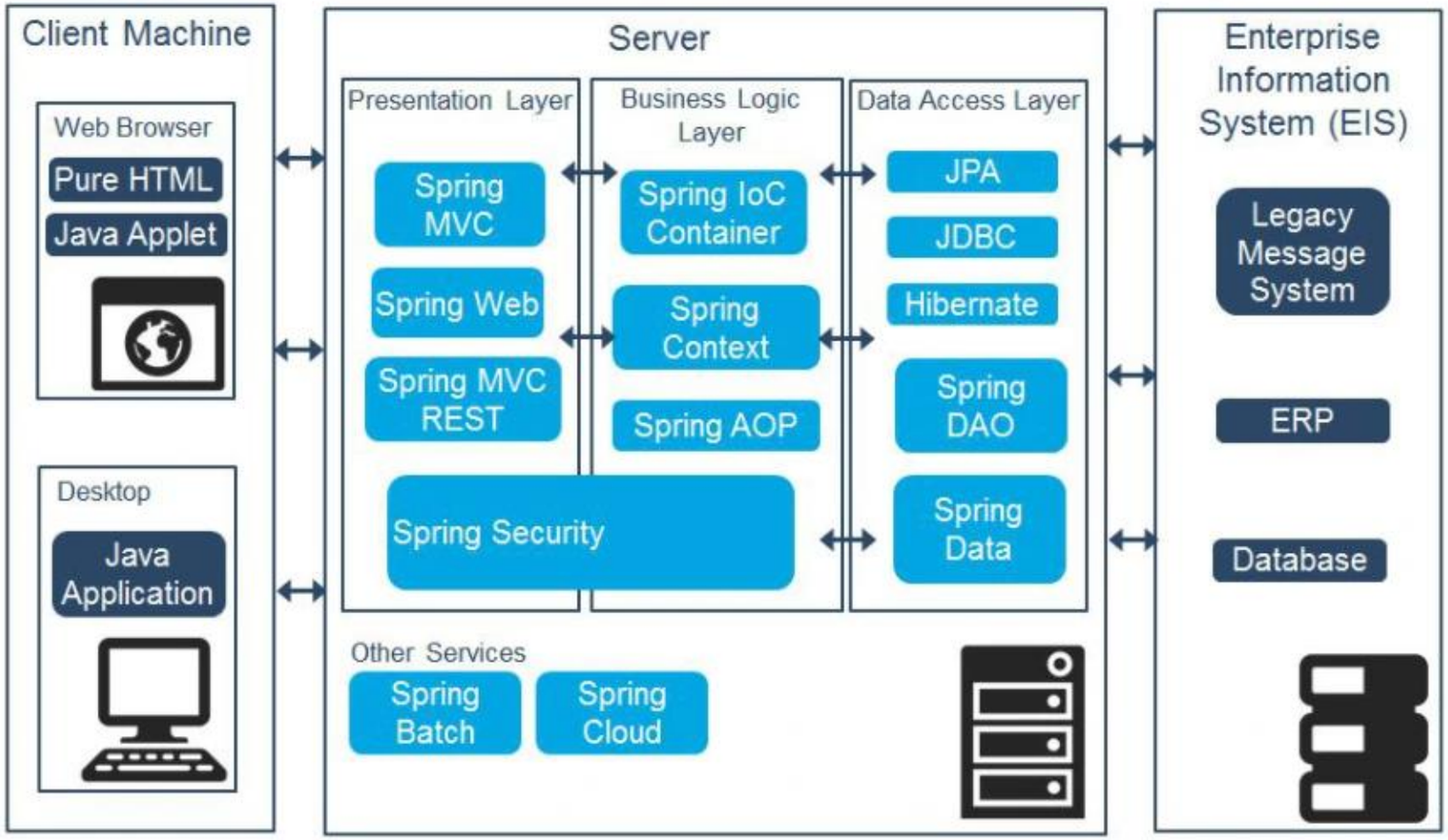
ÜBERSICHT

- JEE am Beispiel CRM-DEMO
- 12 Faktor Apps
- Spring

SPRING – SPRING.IO

- Quelloffenes open-source Java Framework
- Ökosystem aus einem Kern-Framework und mehreren Projekten
- Ursprünglich als leichtgewichtiger Gegenentwurf zu JEE entstanden
  - JEE Architekturen als oft unnötig komplex empfunden
  - Mittlerweile nicht mehr so gesehen - Auch keine direkte Konkurrenz
- Kernkonzepte sind
  - POJOs (plain old java objects)
  - Annotations
  - Dependency Injection
- Die so von Spring verwalteten "Komponenten" werden Spring Beans genannt

SPRING ARCHITEKTUR

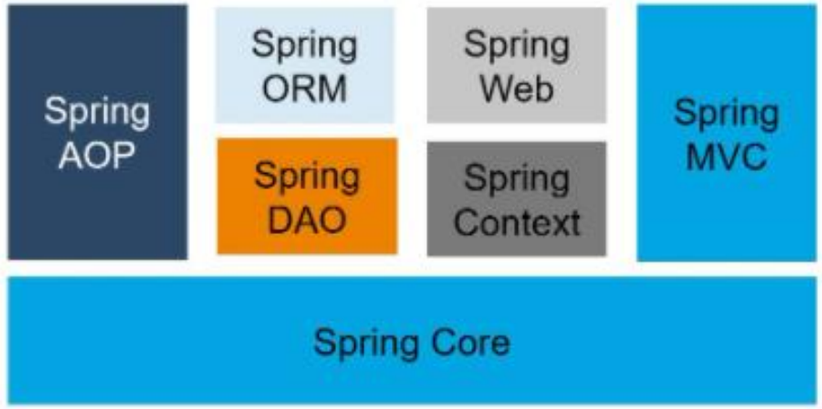


Spring Komponenten/ Technologien

[https://blog.doubleslash.de/jee-vs-spring-gemeinsamkeiten-unterschiede-und-entscheidungskriterien/]

SPRING FRAMEWORK MODULE

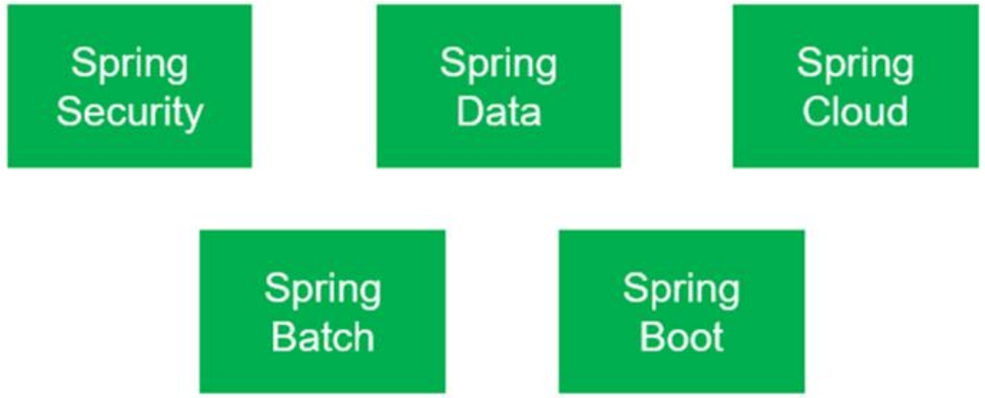
- Zentrales Modul Spring Core mit Dependency Injection Container
- Spring Context erweitert um Validierung, EJB-Integration und Scheduling
- Spring ORM für Integration OR-Mapper wie Hibernate
- Spring Web MVC für Webanwendungen mit MVC und REST-Webservices



Spring Framework Module

[https://blog.doubleslash.de/jee-vs-spring-gemeinsamkeiten-unterschiede-und-entscheidungskriterien/]

SPRING ÖKOSYSTEM MODULE



- Erweitern das Framework
  - Zusätzliche Features
  - Auf Basis des Frameworks
- Spring Boot das wohl bekannteste
  - Vereinfacht die Erstellung von Spring (Web-)Applikationen deutlich
  - Kein Applicationserver benötigt
  - Stattdessen ein embedded Tomcat mitgeliefert
  - Anwendung somit als Self-Contained-JARs auslieferbar
  - Ideale Vorraussetzung für Microservice getriebene Architekturen

[https://blog.doubleslash.de/jee-vs-spring-gemeinsamkeiten-unterschiede-und-entscheidungskriterien/]

ABGRENZUNG JEE / SPRING

- Sowohl JEE als auch Spring bieten ein großes Ökosystem
- Unterschiede verschmelzen zusehends
- Was sich in JEE umsetzen lässt, lässt sich auch in Spring umsetzen
- Wesentlicher Unterschied ist der Verzicht auf einen Applicationserver bei Spring
- Unterschiede in weiteren Punkten gehen in Richtung Vorlieben/Erfahrungen
- Insgesamt überwiegen die Gemeinsamkeiten der beiden Frameworks
- Spring Dokumentation beschreibt das Verhältnis als ergänzend
- Somit keine Konkurrenz im direkten Sinne, auch wenn oft so wahrgenommen

ABGRENZUNG JEE / SPRING

	Java EE	spring
Generell	API, es gibt viele versch. Implementierungen	Konkrete Implementierung
Server	JEE konformer Application Server	Standalone oder Web Server
Dependency Injection	CDI	Spring IoC/ Spring Core
AOP	Interceptor	Spring AOP
Persistence	JPA	JPA (Default: Hibernate), JDBC
UI	JSF2	Spring MVC
WS	JAX-RS, JAX-WS	Spring MVC REST, Spring Web Services, Spring Data REST
Security	Java EE Security	Spring Security
Testing	Arquillian (nur über AppServer)	Spring Testing
Transactions	JTA	JTA/ Spring Data

JEE vs Spring

[https://blog.doubleslash.de/jee-vs-spring-gemeinsamkeiten-unterschiede-und-entscheidungskriterien/]

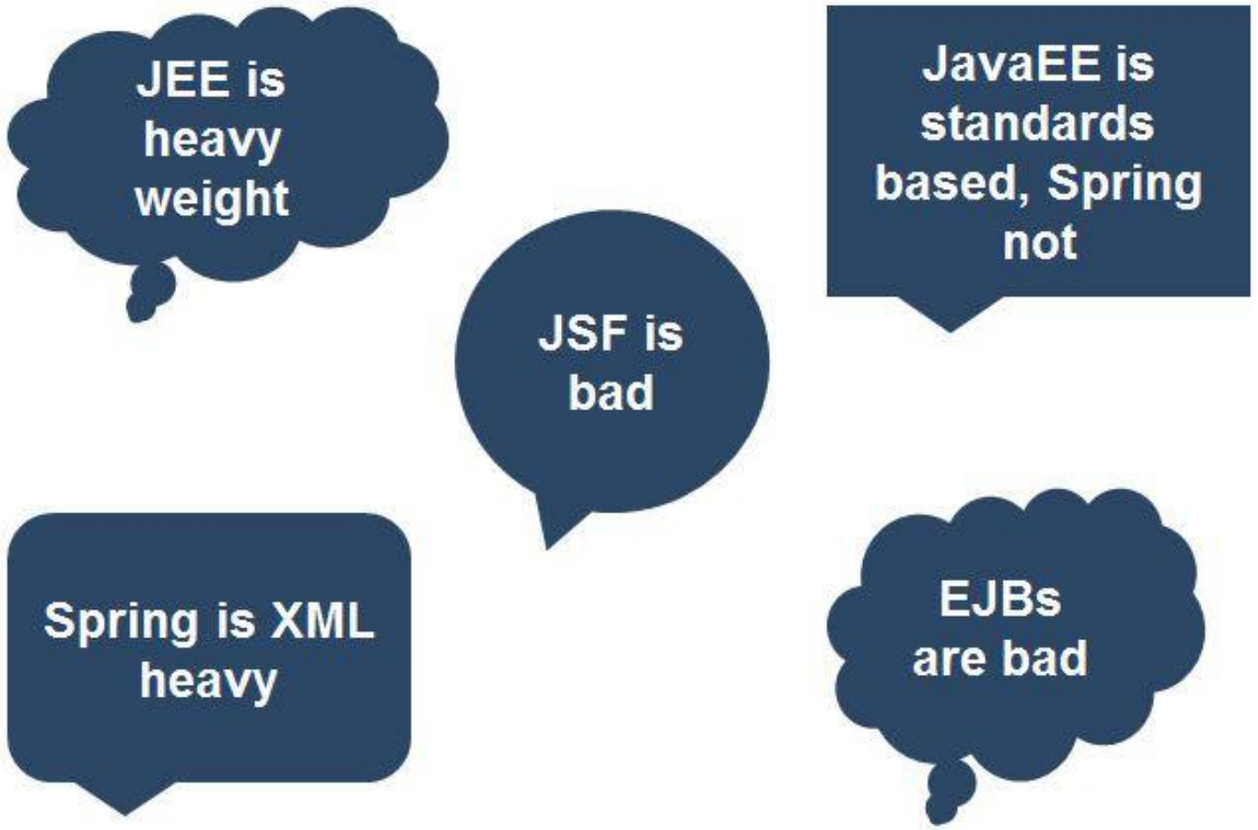
ABGRENZUNG JEE / SPRING

Java EE	spring
(–) Einarbeitung + Konfiguration AppServer	(+) Keine Einarbeitung in AppServer
(–) Aufwändige Einrichtung eines AppServers bei den Entwicklern	(+) Out-of-the-box-Funktion
(–) nutzen Kunden versch. AppServer muss jeder konfiguriert werden	(+) Applikation kann auf x-beliebigem Webserver Out-of-the-box laufen
(+) AppServer bringt Funktionalitäten mit wie Clustering, TimerService	(–) Eigenimplementierung Funktionalitäten oder Web Server nutzen
(–) Testing über AppServer (Arquillian)	(+) Einfacheres Testing
(+) versch. Implementierungen zur Auswahl (+) Standardisiert. Wechsel möglich.	(+) Komponenten austauschbar
(–) Ergänzung neue Features langsamer	(+) Ergänzung neue Features schneller

Vor- und Nachteile Spring vs Java EE

[https://blog.doubleslash.de/jee-vs-spring-gemeinsamkeiten-unterschiede-und-entscheidungskriterien/]

VORURTEILE GEGENÜBER JEE NICHT MEHR ZEITGEMÄß



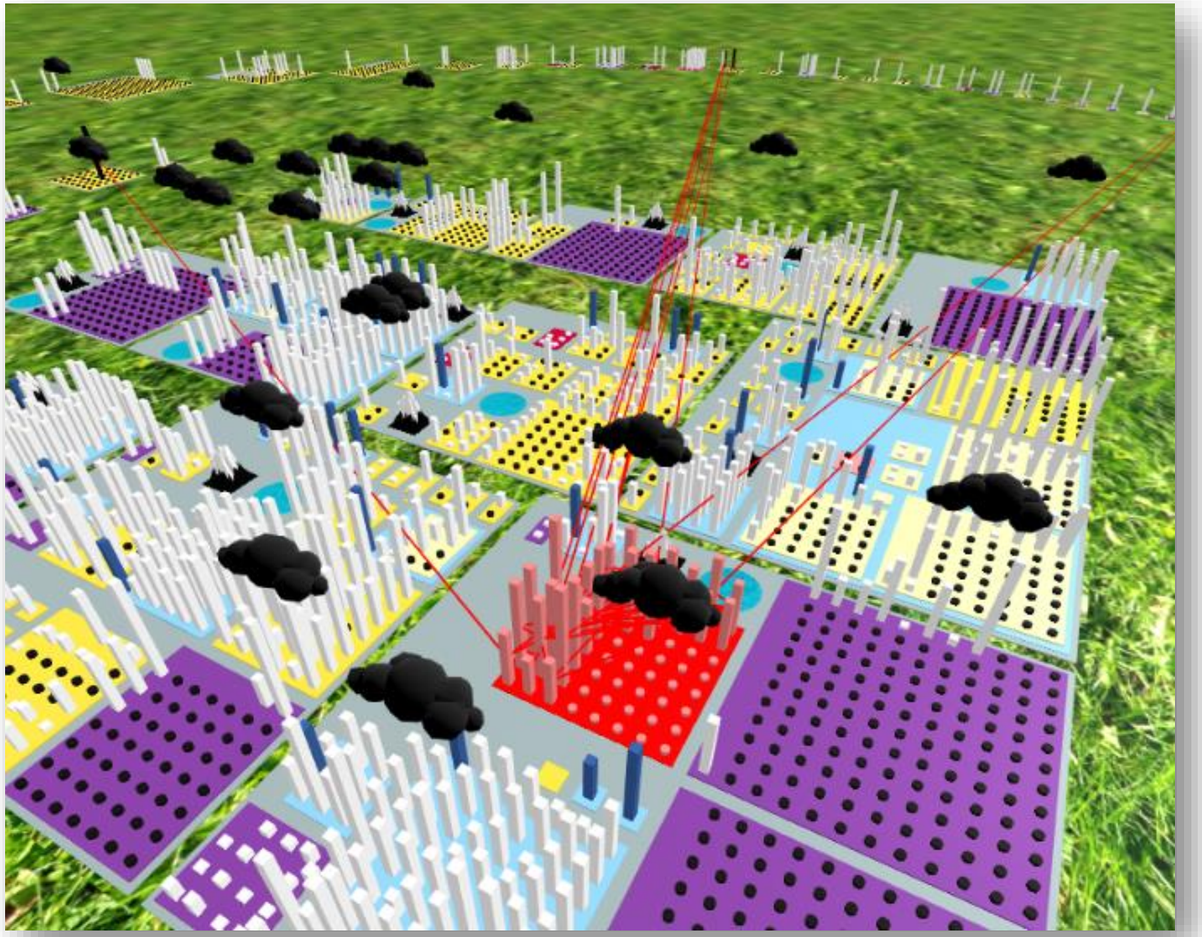
[https://blog.doubleslash.de/jee-vs-spring-gemeinsamkeiten-unterschiede-und-entscheidungskriterien/]

URSPRUNG DER „FALSCHEN“ VORURTEILE

- Ältere Versionen von JEE (Vormals J2EE) deutlich komplexer/umständlicher
- Durch Annotationen beispielsweise aber auch deutlich einfacher geworden
- JEE als „standardisierte“ Spezifikation etwas langsamer für neue Feature
- Was nicht zwingend ein Nachteil sein muss
- Trend zu Microservice getriebenen Architekturen auch in JEE aufgenommen
- Self-Contained-Jars mittels Konfigurierbaren Servern wie Thorntail möglich

UNIVERSITÄT
LEIPZIG

VIELEN DANK!

Dr. Pascal Kovacs
Institut für Wirtschaftsinformatik

pkovacs@uni-leipzig.de
https://www.wifa.uni-leipzig.de/iwi/